**INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA**
**DEPARTMENT OF MECHATRONICS ENGINEERING**

# MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

## SEMESTER 1, 2026/2027

## EXPERIMENT 1: DIGITAL LOGIC DESIGN

## SECTION 1

## GROUP 10



| NO | NAME | MATRIC NUMBER |
|---|---|---|
| 1. | WIBOWOPUTRI ALYA SHAFIQA | 2416728 |
| 2. | NURUL SYIFA BINTI RAZALI | 2416110 |
| 3. | NURUL IZZAH KHALIESA BINTI MOHD NIZAM | 2414774 |
| 4. | SHARIFAH NUR AZZARIENA EMILY BT SYED MOHD AZAM | 2417458 |



**SUBMISSION DATE: 11 OCTOBER 2026**

# ABSTRACT

Digital logic systems process binary signals to control electronic devices based on programmed conditions. This experiment aimed to develop a pushbutton-controlled digital counter using an Arduino Uno and a seven-segment display to demonstrate digital input/output operations and basic programming logic. The circuit was assembled on a breadboard using a seven-segment display, current-limiting resistors, two pushbuttons, and connecting wires. The firmware was developed and uploaded through the Arduino IDE. The increment button was programmed to increase the displayed count from 0 to 9 before returning to zero, while the reset button returned the count to zero. The system's operation was evaluated by observing the seven-segment display, and a separate preliminary test was used to check the pushbutton inputs through the Serial Monitor. A key technical consideration was the display configuration, as the observed connection to 5 V and active-low segment control indicated common-anode operation, despite the common-cathode designation in the task instructions. To improve the system's reliability, software debouncing should be implemented to minimise unintended multiple counts caused by mechanical switch bouncing. The display type should also be verified before assembly to ensure correct segment patterns. Additionally, integrating an I2C LCD could provide clearer count and system-status information, while checking wiring connections and selecting the correct communication port could reduce setup errors. These improvements would enhance the system's reliability, usability, and ease of testing.

# TABLE OF CONTENT


**1.0 INTRODUCTION AND THEORY ..... 4**
**1.1 INTRODUCTION ..... 4**
**1.2 THEORETICAL BACKGROUND ..... 4**
**1.2.1 Digital Logic and Arduino Input/Output ..... 4**
**1.2.2 Seven-Segment Display and Segment Control ..... 4**
**1.2.3 Pushbuttons and Digital Input Signals ..... 4**
**1.2.4 Firmware Logic and Counting Operation ..... 5**
**1.2.5 Preliminary Testing and Computer-Side Software ..... 5**
**1.3 EXPECTED TECHNICAL OUTCOMES ..... 5**
**2.0 MATERIALS AND EQUIPMENT ..... 5**
**3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY ..... 6**
**4.0 METHODOLOGY AND PROGRAM LOGIC ..... 6**
**4.1 Circuit Assembly & Hardware Configuration ..... 6**
**4.2 Programming & Uploading ..... 7**
**4.3 Programming Logic ..... 7**
**4.3 Source Code ..... 7**
**5.0 DATA COLLECTION AND TECHNICAL ANALYSIS ..... 10**
**6.0 RESULTS AND ENGINEERING DISCUSSION ..... 11**
**6.1 RESULTS ..... 11**
**6.2 DISCUSSION ..... 13**
**7.0 CONCLUSION ..... 14**
**8.0 RECOMMENDATIONS ..... 14**
**9.0 REFERENCES ..... 15**
**10.0 APPENDICES ..... 15**
**11.0 ACKNOWLEDGEMENT ..... 16**
**12.0 STUDENT DECLARATION ..... 16**

# 1.0 INTRODUCTION AND THEORY

## 1.1 INTRODUCTION

Digital logic systems are widely used in electronic devices to process input signals and produce specific outputs based on programmed conditions. These systems operate using two basic logic states, HIGH and LOW, which represent binary values of 1 and 0, respectively. By combining digital inputs, logical operations, and output control, a microcontroller can perform various functions, including counting, displaying information, and controlling electronic components. This experiment demonstrates the basic principles of digital logic through the development of a pushbutton-controlled counting system using an Arduino Uno and a seven-segment display.

The Arduino Uno is a microcontroller development board based on the ATmega328P microcontroller. It provides programmable digital input and output pins that allow it to receive electrical signals from external components and control connected devices. In this experiment, two pushbuttons are used to control the counting operation, while a seven-segment display presents the numerical output. The increment button increases the counter value, whereas the reset button returns the counter to zero. The Arduino processes the button inputs according to the programmed conditions and generates the corresponding output signals to illuminate the required display segments.

The main objective of this experiment is to design and test a digital counter that displays digits from 0 to 9 using an Arduino Uno, pushbuttons, and a seven-segment display. The experiment also aims to investigate the relationship between digital input signals, firmware logic, and output control, as well as to understand the function of current-limiting resistors and the use of a computer interface for preliminary input testing.

## 1.2 THEORETICAL BACKGROUND

### 1.2.1 Digital Logic and Arduino Input/Output

Digital logic uses two states, HIGH and LOW, representing binary values 1 and 0. The Arduino Uno, based on the ATmega328P microcontroller, reads digital signals through input pins and controls external components through output pins.

### 1.2.2 Seven-Segment Display and Segment Control

A seven-segment display consists of seven LED segments labelled A to G, which illuminate in different combinations to represent digits from 0 to 9. Displays are classified as common-anode or common-cathode types. Common-anode displays illuminate when the segment control pin is LOW, whereas common-cathode displays illuminate when it is HIGH.

### 1.2.3 Pushbuttons and Digital Input Signals

Two pushbuttons connected to Arduino pins D10 and D11 control the increment and reset functions. External 10 kΩ pull-up resistors connect the inputs to 5 V, while pressing a button connects its input to ground. The program waits for button release and applies short

delays to reduce repeated counting, although it does not implement a dedicated debouncing algorithm.

### 1.2.4 Firmware Logic and Counting Operation

The Arduino firmware defines the pin assignments, initialises the counter at zero, and stores the segment patterns for digits 0–9 in a two-dimensional array. The displayNumber() function uses these patterns to control the display, with LOW illuminating a segment and HIGH switching it off. The main loop() continuously checks the buttons: pressing the increment button increases the count, returning it to zero after 9, while pressing the reset button immediately sets the count to zero.

### 1.2.5 Preliminary Testing and Computer-Side Software

The Arduino IDE is used to write, compile, and upload the firmware through USB. During preliminary testing, a separate program and the IDE's Serial Monitor were used to check the pushbutton input states. The main program also initialises serial communication at 9600 baud and contains commands to print the count and reset messages.

## 1.3 EXPECTED TECHNICAL OUTCOMES

At the end of the experiment, the system is expected to demonstrate a functional 7-segment display controlled by two pushbuttons. Pressing the increment button should increase the displayed value sequentially from 0 to 9, after which the next increment should return the counter to zero. Pressing the reset button should return the display to zero regardless of the current count. The experiment is also expected to demonstrate the relationship between binary logic states and visible output patterns, the use of current-limiting resistors to protect LED segments, and the role of firmware in processing digital inputs and controlling outputs. Successful operation would indicate that the hardware connections and programmed logic work together to achieve the intended counting function.

## 2.0 MATERIALS AND EQUIPMENT

- Arduino Uno board
- Common cathode 7-segment display
- 220Ω resistors (7)
- 10kΩ resistors (2)
- Pushbuttons (2)
- Jumper wires
- Breadboard

## 3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY

1. Common cathode 7-segment display to Arduino Uno:
    - Each segment pin (a, b, c, d,e, f, g) of the display was connected to Arduino Uno digital pins (D2, D3, D4, D5, D6, D7, D8) through 220Ω resistors to limit the current flow.
    - The common cathode pin of the display was connected to one of the Arduino Uno GND pin.
2. Pushbuttons to the Arduino Uno:
    - One leg of the first pushbutton was connected to Arduino Uno pin D10 to act as the increment button.
    - One leg of the second pushbutton was connected to Arduino Uno pin D11 to act as the reset button.
    - The opposite legs of the both pushbuttons were connected to the negative rail of the breadboard.
    - A 10kΩ pull-up resistor was connected to the increase button with one end attached to the corresponding digital input pin D10 and the other end connected to the 5V pin

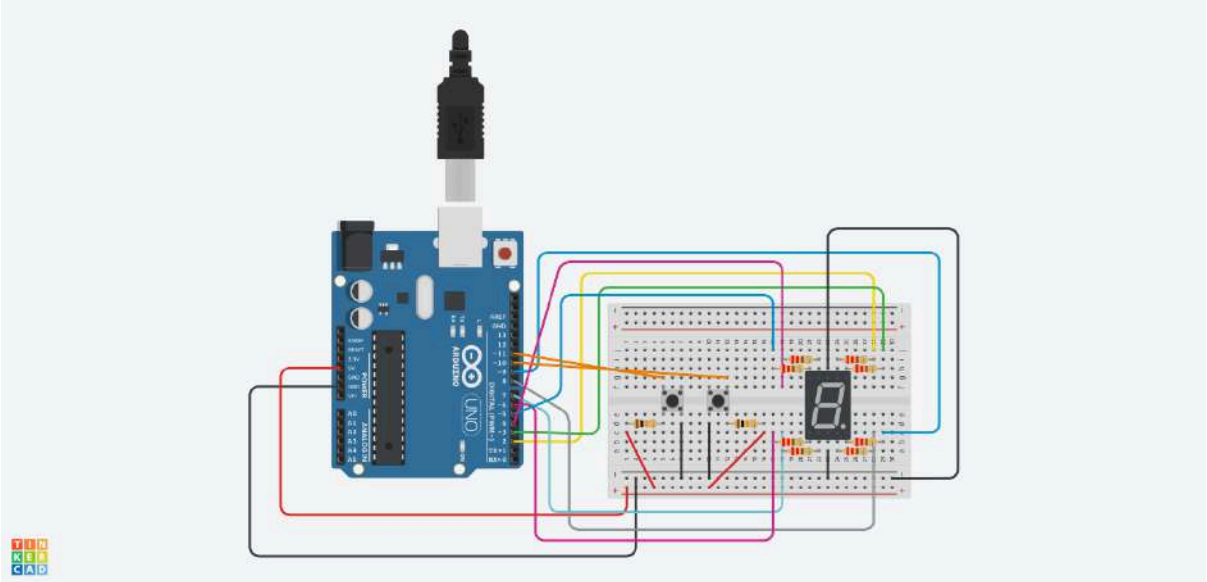


*Figure 3.1 Circuit Design Schematics using Tinkercad*

## 4.0 METHODOLOGY AND PROGRAM LOGIC

### 4.1 Circuit Assembly & Hardware Configuration

- The hardware components were assembled on a breadboard following the standard common-cathode 7 segment display configuration.
- The 7 segment pins ( a to g ) were connected to Arduino digital pins 2 to 8 through individual 220Ω resistors.
- The decimal point pin was connected to digital pin 9.
- The common cathode pin of the 7 segment display was connected directly to one of the ground(GND) pins on the Arduino.

- Two pushbuttons for incrementing and resetting were connected to digital pins 10 and 11 respectively, utilizing external 10kΩ resistors to maintain HIGH state when unpressed and pulling the input to LOW when pressed.

## 4.2 Programming & Uploading

- The Arduino IDE on the computer was used to write and upload the code making sure the correct board and port were selected.
- The code was compiled to make sure there were no syntax errors.
- The code then uploaded to the board

## 4.3 Programming Logic

- The segment pins and push button pins were initialized.
- The counter variable was set to zero.
- The program continuously monitors the button inputs.
- When the increment button is pressed(LOW), the counter increases by 1 and the display is updated.
- When the reset button is pressed(LOW), the counter immediately resists back to 0 and the display shows “0”.
- The displayNumber( ) function controlled the output by turning ON and OFF specific segments according to current count.

## 4.3 Source Code

```
// ========================
// 7-SEGMENT PINS
// ========================
const int a = 2;
const int b = 3;
const int c = 4;
const int d = 5;
const int e = 6;
const int f = 7;
const int g = 8;

const int dp = 9;

// ========================
// PUSH BUTTONS
// ========================
const int increaseButton = 10;
const int resetButton = 11;

int count = 0;
```

```
// ==========================
// NUMBER PATTERNS
// LOW  = ON
// HIGH = OFF
// ==========================

//       a b c d e f g
const byte numbers[10][7] = {
  {0,0,0,0,0,0,1}, // 0
  {1,0,0,1,1,1,1}, // 1
  {0,0,1,0,0,1,0}, // 2
  {0,0,0,0,1,1,0}, // 3
  {1,0,0,1,1,0,0}, // 4
  {0,1,0,0,1,0,0}, // 5
  {0,1,0,0,0,0,0}, // 6
  {0,0,0,1,1,1,1}, // 7
  {0,0,0,0,0,0,0}, // 8
  {0,0,0,0,1,0,0}  // 9
};


// ==========================
// DISPLAY NUMBER
// ==========================
void displayNumber(int number) {

  digitalWrite(a, numbers[number][0]);
  digitalWrite(b, numbers[number][1]);
  digitalWrite(c, numbers[number][2]);
  digitalWrite(d, numbers[number][3]);
  digitalWrite(e, numbers[number][4]);
  digitalWrite(f, numbers[number][5]);
  digitalWrite(g, numbers[number][6]);

  // DP OFF
  digitalWrite(dp, HIGH);
}


// ==========================
// SETUP
// ==========================
```

```
void setup() {

  pinMode(a, OUTPUT);
  pinMode(b, OUTPUT);
  pinMode(c, OUTPUT);
  pinMode(d, OUTPUT);
  pinMode(e, OUTPUT);
  pinMode(f, OUTPUT);
  pinMode(g, OUTPUT);
  pinMode(dp, OUTPUT);

  // Buttons use external 10kΩ pull-up resistors
  pinMode(increaseButton, INPUT);
  pinMode(resetButton, INPUT);

  // Start at 0
  displayNumber(count);

  Serial.begin(9600);
}


// ==========================
// LOOP
// ==========================
void loop() {

  // ==========================
  // INCREASE BUTTON
  // ==========================
  if (digitalRead(increaseButton) == LOW) {

    count++;

    // After 9, go back to 0
    if (count > 9) {
      count = 0;
    }

    displayNumber(count);

    Serial.print("Count: ");
    Serial.println(count);
```

```
    // Wait until button is released
    while (digitalRead(increaseButton) == LOW) {
      delay(10);
    }

    delay(50);
  }


  // ==========================
  // RESET BUTTON
  // ==========================
  if (digitalRead(resetButton) == LOW) {

    count = 0;

    displayNumber(count);

    Serial.println("Reset: 0");

    // Wait until reset button is released
    while (digitalRead(resetButton) == LOW) {
      delay(10);
    }

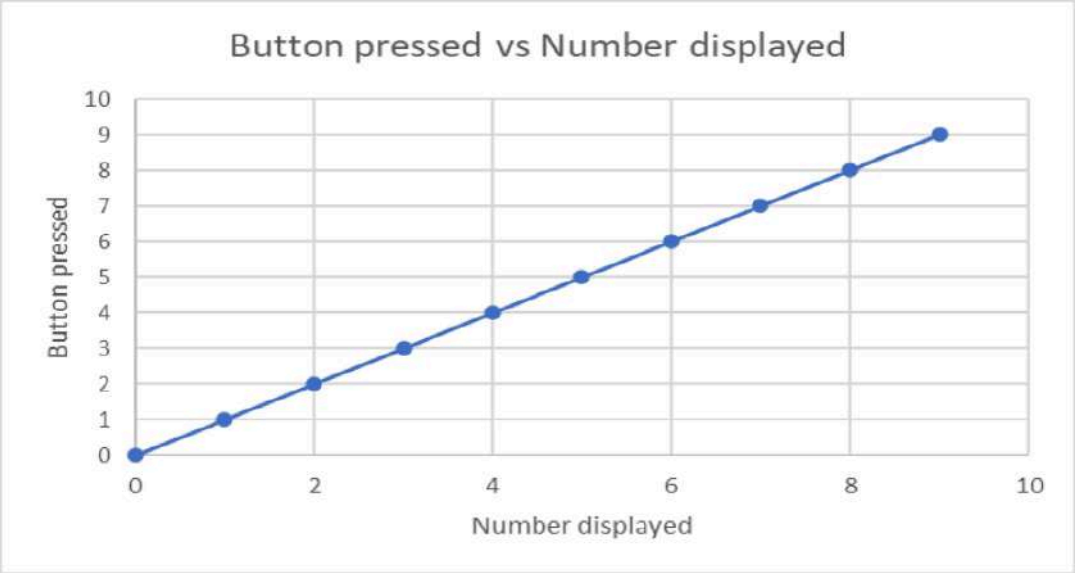
    delay(50);
  }
}
```

## 5.0 DATA COLLECTION AND TECHNICAL ANALYSIS

| BUTTON PRESS | NUMBER DISPLAYED |
|---|---|
| Reset | 0 |
| Increment | 1 |
| Increment | 2 |
| Increment | 3 |
| Increment | 4 |

| | |
|---|---|
| Increment | 5 |
| Increment | 6 |
| Increment | 7 |
| Increment | 8 |
| Increment | 9 |
| Increment | 0 |

*Table 5.1 shows the data collected when the push buttons were pressed during testing*

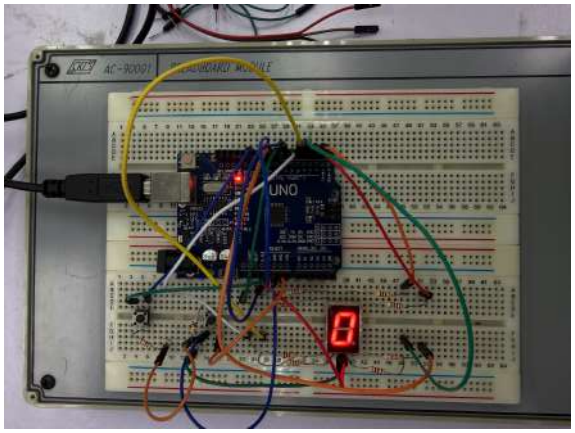



*Graph 5.2 shows the relationship between button pressed and number displayed*

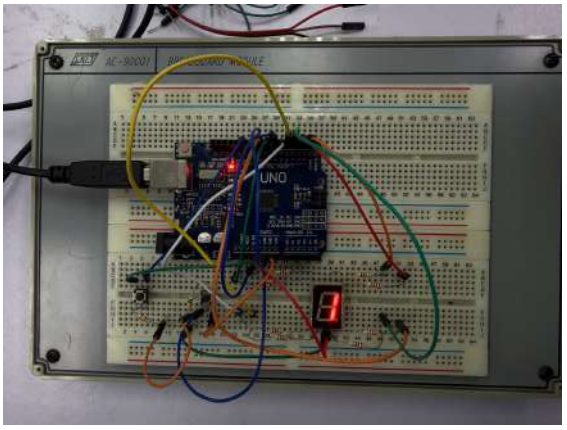
# 6.0 RESULTS AND ENGINEERING DISCUSSION

## 6.1 RESULTS

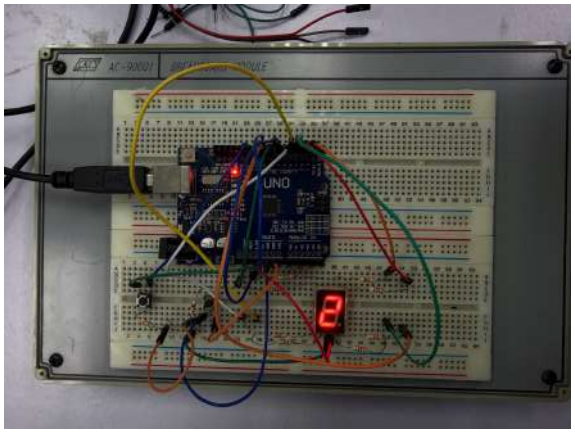
The results shown in the figure below demonstrate the 7-segment display and two pushbuttons successfully display the numbers from 0-9. Each number displayed based on the combination of activated segments. When the increment push button is pressed, the displayed number increases subsequently by one, while on the other hand, when the reset push button is pressed, the display returns to 0. The figures provided clearly show the successful operation of the system functions as intended.



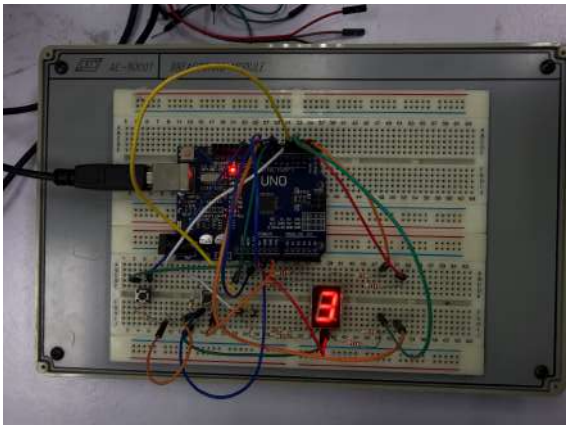

*6.1 The 7-segment display 0*



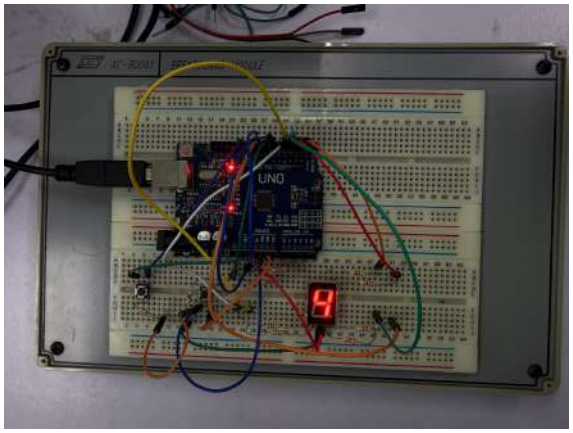

*6.2 The 7-segment display 1*



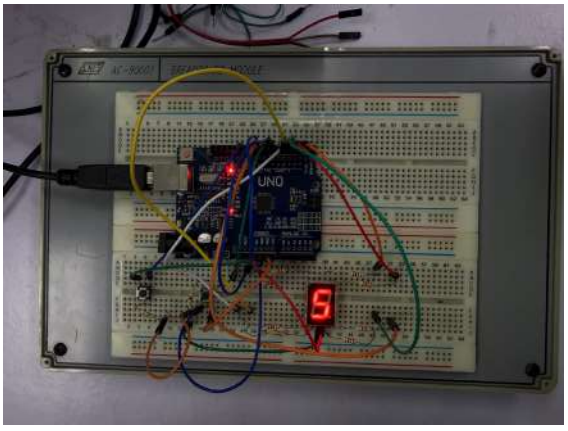

*6.3 The 7-segment display 2*



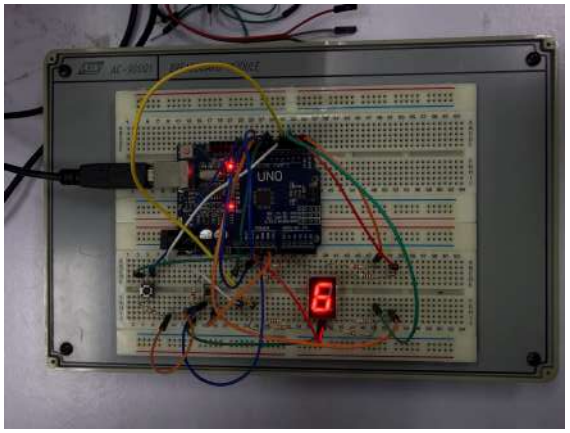

*6.4 The 7-segment display 3*



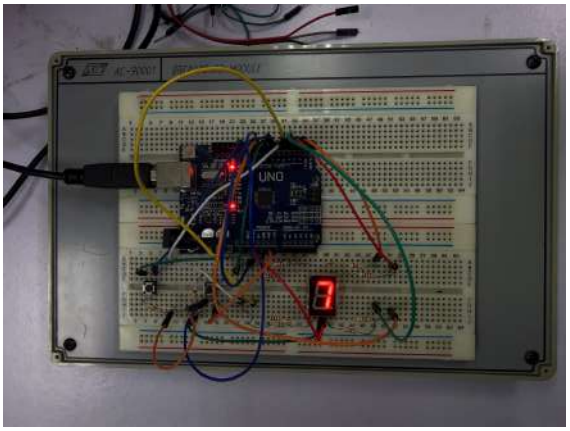

*6.5 The 7-segment display 4*



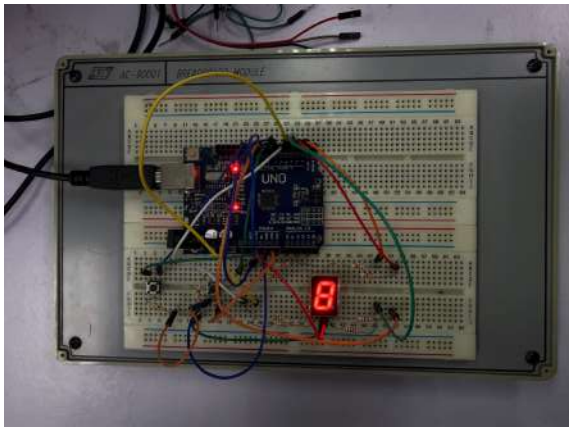

*6.6 The 7-segment display 5*



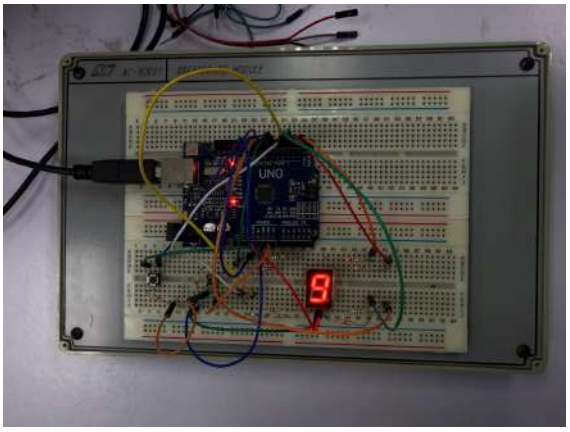
*6.7 The 7-segment display 6*



*6.8 The 7-segment display 7*



*6.9 The 7-segment display 8*



*6.10 The 7-segment display 9*

## 6.2 DISCUSSION

In this experiment, the 7-segment display operated according to the programmed logic where the increment push button successfully increased the displayed number from 0 to 9, while the reset push button returned the display to 0. The system functioned as expected, demonstrating the correct digital interfacing between Arduino Uno and the 7-segment display.

Several issues were encountered during the experiment. Firstly, the type of 7-segment display must be correctly identified. For a common cathode display, the common pins are connected to GND and each segment illuminates when the corresponding output is HIGH. On the other hand, a common anode display has its common pins connected to 5V and each segment illuminates when the corresponding output is LOW. Hence, using the wrong display type or programming logic will result in incorrect digit patterns.

Other than that, another issue encountered was the Arduino COM port not being detected which was caused by selecting the incorrect COM port in the Arduino IDE. This prevented the program from being uploaded to the Arduino board. The problem was resolved by reconnecting the Arduino and selecting the correct COM under Tools then Port.

Question: *How can you interface and l2C LCD with Arduino? Explain the coding principle behind it compared to a 7-segment display and a matrix LED.*

An l2C LCD can be interfaced with an Arduino Uno by using four connections VCC, GND, SDA and SCL. The SDA (Serial Data) line is used to transmit data between the Arduino and the LCD. Besides that, the SCL (Serial Clock) line provides the clock signal to synchronize the communication. For an Arduino Uno, the SDA are connected to A4 while the SCL are connected to the A5 pins. The l2C communication allows multiple devices to communicate using only two signal wires which reduces the number of required Arduino pins compared to a standard parallel LCD connection. Furthermore, the I2C LCD involves using libraries such as LiquidCrystal_I2C . This makes LCD programming easier by providing built-in functions to initialize the LCD, control the cursor position and display text or numerical values. This differs from a 7-segment display which requires the Arduino to control each individual segment (A-G) by applying HIGH or LOW signals to produce specific digits. Moreover, a matrix LED requires a multiplexing method where the rows and columns of LEDs are rapidly scanned to display characters, symbols or patterns. Therefore, the l2C LCD provides a simpler coding method for displaying text while the 7-segment displays are commonly used for numerical displays and a matrix LED is suitable for more complex graphical displays.

## 7.0 CONCLUSION

In conclusion, this experiment was a success as it conforms to the objective and hypothesis. This experiment successfully demonstrated the interfacing of a common cathode 7-segment display with an Arduino Uno using digital output pins and push button inputs. The system was able to display numerical values from 0 to 9 by pushing the increment push button and reset the display back to 0 when the reset push button is pressed. This project taught us practical skills in building circuits and Arduino programming.

In addition, the experiment enhanced troubleshooting skills through the identification of common issues such as incorrect 7-segment display type, incorrect COM port selection and wiring errors. The comparison between the 7-segment display, l2C LCD and matrix LED has further strengthened our understanding of different display methods and their coding principles.

Overall, the experiment provided useful hands-on practice in developing embedded systems and improved our understanding that correct wiring connections, programming and troubleshooting are key to a reliable system.

## 8.0 RECOMMENDATIONS

There are several improvements that can be implemented to enhance the system's performance and reliability. First and foremost, software debouncing can be implemented to the system by adding a short delay , "delay ()" in the code to avoid multiple counts caused by the increment push button bouncing. As a result, the system will have more accurate and stable button inputs.

Besides that, before assembling the circuit, the type 7-segment display should be verified to determine whether it is a common cathode or common anode display. This to ensure the correct digit patterns are displayed. Other than that, add an I2C LCD screen to the system to display the current count and the system status. This can make the system more user friendly and easier to monitor during operation.

Lastly, carefully check the wiring connections and select the correct COM port before uploading the program to minimize wiring errors and communication problems during operation.

## 9.0 REFERENCES

1. Arduino Uno hardware and digital input/output
Arduino UNO R3. Arduino Documentation.
https://docs.arduino.cc/hardware/uno-rev3/

2. Pushbutton operation and debouncing
Arduino Debounce on a pushbutton. Arduino Documentation.
https://docs.arduino.cc/built-in-examples/digital/Debounce/

## 10.0 APPENDICES

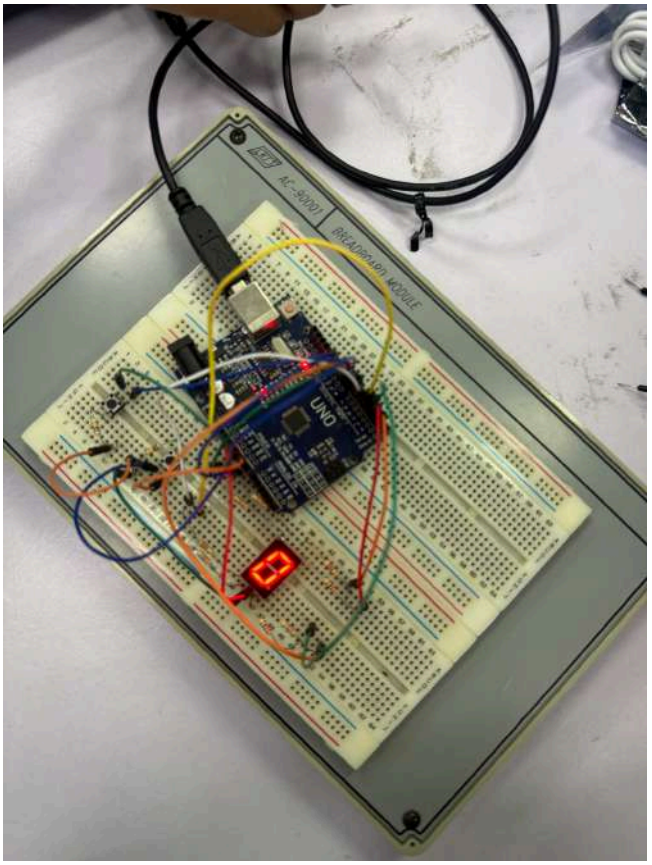



*Figure 10.1 shows the assembled circuit on a breadboard according the designed connection layout*

## 11.0 ACKNOWLEDGEMENT

First and foremost, we would like to express my sincere gratitude to Allah SWT for granting us the opportunity, strength and guidance to learn, complete this laboratory work and gain valuable knowledge throughout this experiment. Other than that, we would like to deeply thank Dr. Ahmad Imran Bin Ibrahim for sharing his valuable knowledge and providing continuous guidance throughout the completion of this laboratory work. His assistance and dedication to teaching have helped us gain a better understanding of the concepts covered in this laboratory. Last but not least, we would also like to extend our heartfelt thanks to our fellow friends for their cooperation and support throughout the completion of this laboratory assignment.

## 12.0 STUDENT DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

*Certificate of Originality and Authenticity*

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

| SIGNATURE | STUDENT DETAILS | CHECKLIST |
|---|---|---|
|  | Name: WIBOWOPUTRI ALYA SHAFIQA<br>Matric No: 2416728 | Read [/ ]<br>Understand [/ ]<br>Agree [/ ] |
|  | Name: NURUL SYIFA BINTI RAZALI<br>Matric No: 2416110 | Read [ / ]<br>Understand [ / ]<br>Agree [ / ] |
|  | Name: NURUL IZZAH KHALIESA BINTI MOHD NIZAM<br>Matric No: 2414774 | Read [/]<br>Understand [/]<br>Agree [/] |

| | | |
|---|---|---|
|  | Name: SHARIFAH NUR AZZARIENA EMILY BT SYED MOHD AZAM<br>Matric No: 2417458 | Read [/]<br>Understand [/]<br>Agree [/] |